

Documentation Technique Phase 2

ChatBot SUP'ONE

Document de référence pour l'équipe projet
expliquant les limites de la Phase 1, les nouveaux concepts
introduits en Phase 2, le fonctionnement du pipeline RAG
(Embeddings, FAISS, LLM) et le passage à React

Table des matières

1	Introduction	2
1.1	Pourquoi ce document existe	2
1.2	Ce que vous allez comprendre	2
1.3	Public cible	2
2	Ce qu'on avait en Phase 1	3
2.1	L'architecture en quelques mots	3
2.2	Ce qui fonctionnait bien	3
2.3	Ce qui posait problème	3
3	Les limites qui ont motivé le passage à la Phase 2	4
3.1	Limite 1 — Le système comparait des mots, pas du sens	4
3.2	Limite 2 — Les réponses étaient figées	4
3.3	Limite 3 — Impossible de croiser plusieurs sources	4
3.4	Limite 4 — Aucune mémoire entre les messages	4
3.5	Limite 5 — Fragilité face aux variantes de langage	4
4	Les concepts clés de la Phase 2	5
4.1	Les Embeddings — lire le sens, pas les mots	5
4.2	FAISS — chercher vite parmi des milliers de vecteurs	5
4.3	Le LLM — rédiger une réponse naturelle	6
4.4	Le RAG — le nom de la stratégie globale	6
5	Le flux complet — de la question à la réponse	7
5.1	Vue d'ensemble	7
5.2	Temps 1 — La préparation de l'index	7
5.3	Temps 2 — Le traitement de chaque question	9
5.4	Schéma du flux complet	10
6	Ce qui change dans le projet	11
6.1	Django ne disparaît pas — son rôle se réduit	11
6.2	Les nouveaux fichiers qui entrent en jeu	11
6.3	L'endpoint principal change de comportement	11
7	Migration de l'interface : de la PWA à React	12
7.1	Ce qu'était la PWA en Phase 1	12
7.2	Pourquoi passer à React	12
7.3	Ce qui disparaît et ce qui reste	12
8	Points clés à retenir	14

1 Introduction

1.1 Pourquoi ce document existe

Le Club Informatique SUP'PTIC a livré une première version fonctionnelle du chatbot en Phase 1. Ce premier système répondait aux questions des étudiants en cherchant dans une base de données la question la plus proche de celle posée. Il a bien fonctionné — mais l'utilisation réelle a mis en évidence des limites concrètes qu'on ne pouvait pas contourner sans changer d'approche.

Ce document explique honnêtement ce qui n'allait pas, pourquoi on a choisi une nouvelle architecture, et comment elle fonctionne. Il n'y a pas besoin d'être développeur pour le lire — chaque concept technique est expliqué avec des mots simples et des exemples tirés du projet.

1.2 Ce que vous allez comprendre

À la fin de ce document, vous saurez répondre à ces questions :

1. Pourquoi la Phase 1 avait des limites qu'on ne pouvait pas corriger juste en ajoutant plus de données ?
2. C'est quoi concrètement un embedding, FAISS, un LLM, le RAG ?
3. Comment une question posée par un étudiant devient une réponse affichée à l'écran, étape par étape ?
4. Qu'est-ce qui change dans le projet par rapport à ce qu'on avait ?

1.3 Public cible

Ce document s'adresse à tous les membres du club — développeurs, data curators, designers, nouveaux arrivants. Aucune connaissance préalable en intelligence artificielle n'est requise.

2 Ce qu'on avait en Phase 1

2.1 L'architecture en quelques mots

La Phase 1 était construite autour de **Django** et d'une base de données **SQLite**. Les données — questions, réponses, catégories — étaient stockées dans des fichiers JSON, puis importées dans la base via un script Python. Une fois en base, Django s'occupait de tout : stocker les FAQ, calculer les vecteurs de recherche, répondre aux requêtes de l'interface.

L'algorithme de recherche utilisé était le **TF-IDF** combiné à la **similarité cosinus**. En résumé : chaque question de la base était transformée en une suite de nombres (un vecteur) représentant les mots qu'elle contient. Quand un étudiant posait une question, cette question était elle aussi transformée en vecteur, et le système cherchait dans la base le vecteur le plus proche. Il retournait ensuite la réponse associée.

Ce que faisait le système à chaque question

1. L'étudiant envoie sa question via l'interface web
2. Django transforme la question en vecteur TF-IDF
3. Le système compare ce vecteur à tous ceux stockés en base
4. Il trouve la question la plus proche et retourne sa réponse
5. La réponse est affichée — telle quelle, sans modification

2.2 Ce qui fonctionnait bien

Le système Phase 1 avait de vraies qualités. Il était rapide, stable, déployé et utilisable. L'API Django répondait correctement, l'interface web était fonctionnelle, et le système de feedback permettait de collecter les retours des utilisateurs. Pour des questions formulées exactement comme celles stockées en base, les résultats étaient très bons.

2.3 Ce qui posait problème

Le problème fondamental de TF-IDF, c'est qu'il compare des **mots**, pas des **idées**. Deux phrases peuvent vouloir dire la même chose avec des mots complètement différents — et TF-IDF les traitera comme étrangères l'une à l'autre.

Le problème central de la Phase 1

Un étudiant tape : *¿ C'est combien pour s'inscrire ? ¿*

La base contient : *¿ Quels sont les frais de scolarité ? ¿*

Ces deux phrases veulent dire la même chose. Mais elles n'ont **aucun mot en commun**.

Le score TF-IDF est donc proche de zéro, et le chatbot répond à côté — ou pire, dit qu'il ne sait pas — alors que la réponse est bien là, dans la base.

Ce problème ne pouvait pas être résolu en ajoutant plus de données. Même avec 10 000 questions en base, si l'étudiant utilisait des mots différents de ceux stockés, le système échouait. C'est une limite **structurelle** de l'algorithme, pas un manque de contenu.

3 Les limites qui ont motivé le passage à la Phase 2

Voici les cinq limites concrètes observées après le déploiement de la Phase 1.

3.1 Limite 1 — Le système comparait des mots, pas du sens

C'est la limite la plus importante. TF-IDF ne comprend pas le sens des phrases. Il compte les mots, calcule leur fréquence, et cherche des correspondances lexicales. Deux synonymes, deux formulations différentes d'une même idée, une faute de frappe — et le résultat devient mauvais. Un étudiant qui écrit *ń tarif scolarité ž*, *ń combien ça coûte ž* ou *ń prix inscription ž* pour demander la même chose obtiendra des résultats différents selon la formulation, et pas toujours le bon.

3.2 Limite 2 — Les réponses étaient figées

Le chatbot ne générait rien. Il copiait-collait directement le contenu du champ **responses** de l'objet JSON le mieux classé. Peu importe comment l'étudiant avait formulé sa question, la réponse était toujours identique, rédigée d'une seule façon. C'est fonctionnel, mais ça donne un résultat robotique — et surtout, ça ne s'adapte pas au contexte de la question posée.

3.3 Limite 3 — Impossible de croiser plusieurs sources

Certaines questions touchent à plusieurs sujets à la fois. *ń Comment rejoindre le club informatique et quels sont les frais ? ž* nécessite des informations qui viennent de deux objets JSON différents. En Phase 1, le système retournait toujours **une seule réponse** — la meilleure correspondance unique. Tout le reste était ignoré. Les réponses à des questions un peu complexes étaient donc forcément incomplètes.

3.4 Limite 4 — Aucune mémoire entre les messages

Chaque question était traitée de façon totalement indépendante. Si un étudiant demandait d'abord *ń Quelles sont les filières disponibles ? ž*, puis enchaînait avec *ń Et les frais pour celle-là ? ž*, le chatbot ne savait pas de quelle filière il parlait. Il repartait de zéro à chaque message, sans aucun contexte de ce qui venait d'être échangé.

3.5 Limite 5 — Fragilité face aux variantes de langage

Les abréviations, les fautes de frappe courantes, les expressions familières ou les mots en français perturbaient le calcul TF-IDF. *ń suptptic ž*, *ń frais scol ž*, *ń comment register ž* — autant de formulations réelles que des étudiants utilisent et qui faisaient échouer la recherche malgré la présence de la réponse en base.

Ce qu'il faut retenir

Ces cinq limites ne venaient pas d'un manque de données ni d'un bug dans le code. Elles venaient du fait que **TF-IDF compare des mots, pas des intentions**. Quelle que soit la quantité de données ajoutée, on ne pouvait pas résoudre ce problème sans changer d'algorithme. C'est pourquoi la Phase 2 introduit une approche différente.

4 Les concepts clés de la Phase 2

La Phase 2 introduit quatre notions nouvelles qui reviennent constamment dans les discussions du projet. Il est important de bien les distinguer car elles sont souvent confondues.

4.1 Les Embeddings — lire le sens, pas les mots

En Phase 1, TF-IDF transformait une phrase en vecteur en comptant les mots qu'elle contient. Deux phrases sans mot en commun donnaient deux vecteurs très éloignés, même si elles voulaient dire la même chose.

Les **embeddings** font quelque chose de fondamentalement différent. Ils transforment une phrase en vecteur en se basant sur son **sens**. Ils ont été produits par un modèle entraîné sur des centaines de millions de phrases pour apprendre que *¡ frais de scolarité ¿* et *¡ combien coûte l'inscription ¿* expriment la même intention — et donc produisent des vecteurs proches.

La même question avec les deux algorithmes

Question posée : *¡ C'est combien pour s'inscrire ? ¿*

Question en base : *¡ Quels sont les frais de scolarité ? ¿*

Avec TF-IDF (Phase 1) : score ≈ 0.00 — aucun mot en commun

Avec Embeddings (Phase 2) : score ≈ 0.82 — même intention détectée

Le modèle utilisé s'appelle **MiniLM-L6-v2**. Ce n'est pas du code que l'équipe écrit — c'est une bibliothèque Python (`pip install sentence-transformers`) que l'on installe et que l'on utilise. Elle transforme n'importe quel texte en un vecteur de **384 nombres**.

Analogie simple

Imaginez que chaque phrase reçoit des coordonnées GPS dans un espace à 384 dimensions. Deux phrases qui veulent dire la même chose se retrouvent proches géographiquement, même si leurs mots sont différents. TF-IDF, lui, regardait uniquement les mots — pas la position dans cet espace.

4.2 FAISS — chercher vite parmi des milliers de vecteurs

Une fois que toutes les questions de la base ont été transformées en vecteurs par MiniLM, il faut pouvoir chercher très rapidement lequel est le plus proche d'une nouvelle question. C'est exactement ce que fait **FAISS**.

FAISS est une bibliothèque développée par Meta (Facebook), installée via `pip install faiss-cpu`. Elle stocke tous les vecteurs dans un fichier `index.bin` et permet de retrouver en quelques millisecondes les cinq vecteurs les plus proches d'un vecteur donné, même parmi des dizaines de milliers.

FAISS ne sait pas ce que veulent dire les vecteurs. Il sait juste les comparer très vite. C'est le rôle de `metadata.json` de faire le lien entre un numéro de vecteur et la réponse correspondante dans les fichiers JSON.

Ce que FAISS reçoit et ce qu'il retourne

Entrée : le vecteur de la question posée par l'étudiant

Sortie : les numéros des 5 vecteurs les plus proches

Exemple : [42, 871, 304, 1205, 2087]

`metadata.json` est alors consulté pour retrouver la réponse associée à chacun de ces numéros.

4.3 Le LLM — rédiger une réponse naturelle

En Phase 1, une fois la meilleure correspondance trouvée, le système copiait-collait la réponse telle quelle depuis la base. En Phase 2, on introduit un **LLM** (Large Language Model) — un modèle d'intelligence artificielle capable de lire un texte et de rédiger une réponse naturelle.

Le LLM utilisé s'appelle **Phi-3 Mini**. Il tourne entièrement sur la machine locale via un outil appelé **Ollama** — sans connexion internet, sans envoyer de données à l'extérieur.

Concrètement, le LLM reçoit un message structuré contenant : la question posée par l'étudiant, et les cinq réponses récupérées par FAISS. Il rédige alors une réponse fluide et naturelle à partir de ces informations.

Le LLM ne devine rien — il s'appuie sur vos données

Le LLM ne puise pas dans ses propres connaissances générales. Il ne peut répondre qu'avec ce qu'on lui donne. Si FAISS ne trouve aucune correspondance pertinente, le système répond : *¡ Je n'ai pas cette information — contactez le secrétariat. ¢*

C'est une garantie importante : le chatbot ne peut pas inventer des informations sur SUP'TIC.

4.4 Le RAG — le nom de la stratégie globale

RAG signifie *Retrieval Augmented Generation*. Ce n'est pas une bibliothèque, pas un outil à installer. C'est le nom qu'on donne à la stratégie qui combine les trois éléments précédents :

1. **Retrieval** — on *récupère* les réponses les plus pertinentes grâce aux embeddings et à FAISS
2. **Augmented** — on *enrichit* le contexte donné au LLM avec ces réponses
3. **Generation** — le LLM *génère* une réponse naturelle à partir de ce contexte

Le RAG est la recette. MiniLM, FAISS et Phi-3 sont les ingrédients. Le code Django de l'équipe est le cuisinier qui suit la recette.

5 Le flux complet — de la question à la réponse

5.1 Vue d'ensemble

Le système Phase 2 fonctionne en deux temps bien distincts. Le premier est un travail préparatoire fait une seule fois par l'équipe technique. Le second se répète automatiquement à chaque message d'un étudiant.

Les deux temps du système

Temps 1 — Préparation (une seule fois)

Les fichiers JSON sont lus, vectorisés et indexés.

Le système est prêt à recevoir des questions.

Temps 2 — Réponse (à chaque message)

La question est vectorisée, les réponses proches sont trouvées, le LLM rédige et l'interface affiche.

5.2 Temps 1 — La préparation de l'index

Cette étape est invisible pour l'utilisateur final. Elle est exécutée par l'équipe technique chaque fois que de nouveaux contenus sont ajoutés aux fichiers JSON.

Étape 1 — Lecture des fichiers JSON

Le script Python parcourt tous les fichiers JSON du dossier `data/`. Pour chaque objet JSON, il extrait tous les éléments de la liste `examples` — ce sont les différentes façons de formuler une question sur un même sujet.

Étape 2 — Vectorisation par MiniLM

Chaque exemple est envoyé à MiniLM qui le transforme en un vecteur de 384 nombres. Un objet JSON avec 8 exemples produit donc 8 vecteurs distincts — mais tous les 8 pointent vers la même réponse, puisqu'ils viennent du même objet.

Étape 3 — Construction de l'index FAISS

Tous les vecteurs produits sont stockés dans `index.bin`. En parallèle, `metadata.json` est écrit : il associe chaque numéro de vecteur à la réponse et aux informations (catégorie, source) de l'objet JSON dont il est issu.

Ce que contient metadata.json pour 3 vecteurs du même objet

```
{ "vecteur_id": 0,  
  "intent": "faq_frais_0",  
  "example": "C'est combien pour s'inscrire ?",  
  "response": "Les frais sont de 500 000 FCFA par an...",  
  "categorie": "Admissions", "source": "e-supptic.cm" },  
  
{ "vecteur_id": 1,  
  "intent": "faq_frais_0",  
  "example": "Quels sont les frais de scolarite ?",  
  "response": "Les frais sont de 500 000 FCFA par an...",  
  "categorie": "Admissions", "source": "e-supptic.cm" },  
  
{ "vecteur_id": 2,  
  "intent": "faq_frais_0",  
  "example": "Tarif inscription SUP'PTIC",  
  "response": "Les frais sont de 500 000 FCFA par an...",  
  "categorie": "Admissions", "source": "e-supptic.cm" }
```

Les vecteurs 0, 1 et 2 ont des exemples différents mais la même réponse — ils viennent du même objet JSON.

5.3 Temps 2 — Le traitement de chaque question

Un étudiant tape : *¿ C'est combien pour s'inscrire ? ¿*

Étape 1 — Réception par Django

L'interface envoie la question à l'API Django via une requête HTTP.

Étape 2 — Vectorisation de la question

MiniLM transforme la question en un vecteur de 384 nombres, exactement comme il l'a fait pour les exemples lors de la préparation.

Étape 3 — Recherche dans FAISS

FAISS compare ce vecteur à tous ceux stockés dans `index.bin` et retourne les 5 numéros les plus proches.

Étape 4 — Vérification de la pertinence

Chaque résultat a un score entre 0 et 1. Trois cas possibles :

- Score < 0.30 : aucun résultat RAG pertinent — **le système bascule automatiquement sur le moteur TF-IDF de la Phase 1** qui répond comme avant. L'étudiant obtient toujours une réponse, sans voir la différence.
- Score entre 0.30 et 0.55 : résultat approximatif — la réponse sera précédée d'un avertissement
- Score > 0.55 : bonne correspondance — le LLM rédige normalement

Étape 5 — Construction du message pour le LLM

Le système prépare un message structuré qui contient : le rôle du chatbot, les 5 réponses récupérées par FAISS avec leurs métadonnées, et la question originale de l'étudiant. La dernière instruction est toujours : *¿ Réponds uniquement à partir des informations fournies. ¿*

Étape 6 — Génération par Phi-3 Mini

Phi-3 lit le message et rédige une réponse mot par mot. La réponse est envoyée en **streaming** : chaque mot est transmis à l'interface dès qu'il est produit, ce qui donne l'impression que le chatbot tape en temps réel.

Étape 7 — Affichage

L'interface affiche la réponse progressivement. En dessous, les 3 meilleures sources sont affichées avec leur catégorie et leur score, pour que l'étudiant sache d'où vient l'information.

Résultat affiché :

¿ Les frais de scolarité à SUP'PTIC s'élèvent à 500 000 FCFA par an. Ce montant couvre l'ensemble de la formation. Des facilités de paiement peuvent être demandées à la scolarité. ¿

Sources : Admissions (0.91) | e-supptic.cm

5.4 Schéma du flux complet

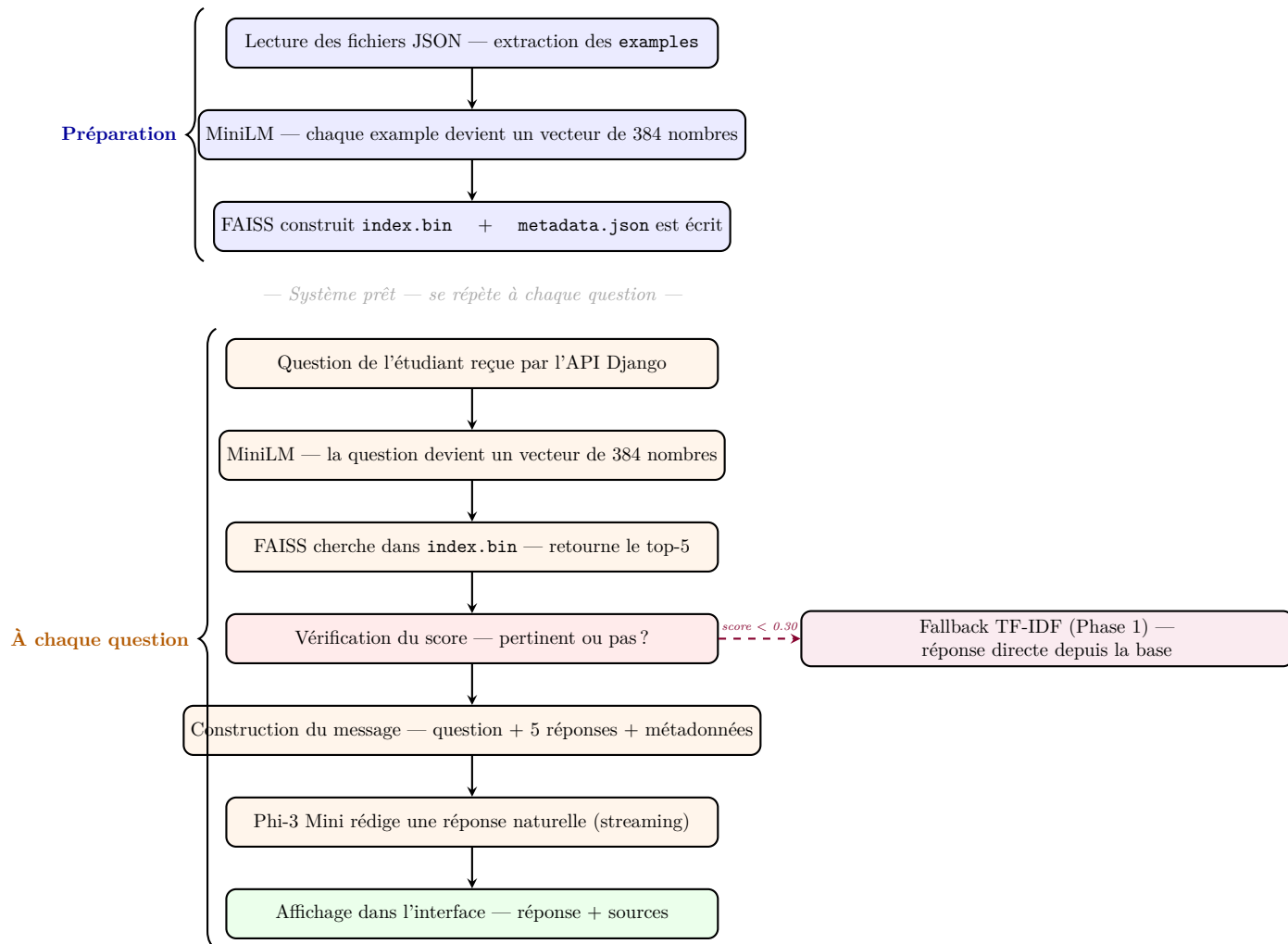


FIGURE 1 – Flux complet du système RAG — Phase 2

6 Ce qui change dans le projet

6.1 Django ne disparaît pas — son rôle se réduit

En Phase 1, Django était le cur du système : il stockait les FAQ, calculait les vecteurs TF-IDF, et faisait la recherche. Tout passait par la base SQLite.

En Phase 2, la recherche et la génération de réponses sont prises en charge par FAISS et Phi-3. Django conserve ce qu'il fait naturellement bien : gérer les utilisateurs, enregistrer les feedbacks, exposer les endpoints de l'API, et afficher les statistiques.

Responsabilité	Phase 1	Phase 2
Stocker les FAQ	Base Django/SQLite	Fichiers JSON sur le disque
Stocker les vecteurs	Table <code>FAQVector</code> en base	<code>index.bin</code> (FAISS)
Faire la recherche	TF-IDF sur la base	MiniLM + FAISS
Produire la réponse	Copier depuis la base	Phi-3 Mini génère
Fallback si RAG insuffisant	<i>n/a</i>	TF-IDF Phase 1 — bascule automatique si score FAISS < 0.30
Feedbacks	Base Django	Base Django (inchangé)
Utilisateurs	Base Django	Base Django (inchangé)
Statistiques	Base Django	Base Django (inchangé)
API REST	Django REST Framework	Django REST Framework (inchangé)

TABLE 1 – Répartition des responsabilités — Phase 1 vs Phase 2

6.2 Les nouveaux fichiers qui entrent en jeu

Deux fichiers nouveaux font leur apparition dans le projet :

- `index.bin` — c'est l'index FAISS. Il contient tous les vecteurs générés à partir des exemples JSON. C'est lui que FAISS consulte lors de chaque recherche. Il est régénéré chaque fois qu'on ajoute de nouveaux contenus aux fichiers JSON.
- `metadata.json` — c'est l'annuaire qui fait le lien entre un numéro de vecteur dans FAISS et la réponse correspondante dans les fichiers JSON. Sans lui, FAISS retournerait des numéros sans pouvoir les associer à du contenu.

6.3 L'endpoint principal change de comportement

L'endpoint `POST /api/chatbot/ask/` existait déjà en Phase 1. En Phase 2, son comportement interne change complètement : au lieu d'interroger la base SQLite avec TF-IDF, il lance le pipeline RAG — vectorisation de la question, recherche FAISS, construction du message, appel à Ollama, streaming de la réponse.

Deux nouveaux endpoints font leur apparition :

- `GET /api/sources/` — retourne les sources JSON utilisées pour produire la dernière réponse, pour affichage dans l'interface
- `POST /api/reload-index/` — recharge `index.bin` en mémoire sans redémarrer Django, utile après l'ajout de nouveaux contenus JSON

7 Migration de l'interface : de la PWA à React

7.1 Ce qu'était la PWA en Phase 1

En Phase 1, l'interface utilisateur était une **Progressive Web App** (PWA) développée en HTML, CSS et JavaScript vanilla. Elle était servie par un petit serveur Python indépendant (`start_server.py`) sur le port 8080, séparé du backend Django. Un `service-worker.js` lui permettait de fonctionner partiellement hors-ligne et d'être installable sur mobile.

Cette approche était simple à mettre en place et fonctionnelle pour une première version. Mais elle montrait ses limites dès que l'interface devenait plus complexe : gestion d'état dispersée dans le DOM, code JavaScript difficile à structurer au fil des nouvelles fonctionnalités, réutilisation des composants quasi impossible.

7.2 Pourquoi passer à React

La Phase 2 introduit plusieurs nouveautés côté interface qui auraient été très difficiles à gérer proprement en JavaScript vanilla :

- **Le streaming de réponse** — afficher les mots du LLM au fur et à mesure qu'ils arrivent nécessite une gestion d'état réactive. En React, un simple `useState` suffit ; en JS vanilla, cela demande une manipulation du DOM fastidieuse et fragile.
- **L'affichage des sources** — chaque réponse est accompagnée de 3 sources avec catégorie et score. Ce sont des composants répétables, exactement le cas d'usage pour lequel React a été conçu.
- **La gestion de l'historique de conversation** — maintenir et afficher une liste de messages avec états différents (en cours, répondu, erreur) est naturel en React grâce à la notion de composants et de props.
- **L'indicateur de confiance** — afficher visuellement le niveau de pertinence d'une réponse (score FAISS) sous forme de jauge ou de badge coloré est trivial en JSX, laborieux en HTML/JS manuel.

Ce que React change fondamentalement

En PWA vanilla, l'interface *manipule* le DOM pour refléter l'état de l'application. En React, l'interface *est* une fonction de l'état — on met à jour l'état, React s'occupe du rendu. Pour une interface de chat avec des données dynamiques qui arrivent en streaming, c'est un gain de fiabilité et de lisibilité considérable.

7.3 Ce qui disparaît et ce qui reste

La migration ne repart pas de zéro. La logique métier reste intégralement côté Django — React ne remplace que la couche d'affichage.

Élément	Phase 1 (PWA)	Phase 2 (React)
Technologie	HTML/CSS/JS vanilla	React (JSX + hooks)
Serveur frontend	<code>start_server.py</code> sur port 8080	<code>npm start</code> / Vite sur port 3000
Gestion d'état	Variables JS + manipulation DOM	<code>useState</code> , <code>useEffect</code>
Composants	Fonctions JS manuelles	Composants React réutilisables
Streaming LLM	EventSource / fetch manuel	Hook dédié avec mise à jour réactive
Offline	Service Worker + cache	Non prioritaire en Phase 2
Installation mobile	Manifest PWA	Non prioritaire en Phase 2
Communication API	<code>fetch()</code> vers Django	<code>fetch()</code> vers Django (inchangé)

TABLE 2 – Comparaison interface — Phase 1 (PWA) vs Phase 2 (React)

Ce que ça change pour les autres équipes

Pour l'équipe backend et data, **rien ne change** : l'API Django REST reste identique, les endpoints sont les mêmes. Seule l'équipe frontend travaille différemment — elle passe de fichiers `.html/.js` à des composants `.jsx` organisés dans un projet React. Les fonctionnalités PWA (mode offline, installation sur mobile) pourront être réintégrées ultérieurement via des bibliothèques React dédiées si le besoin s'en fait sentir.

8 Points clés à retenir

Ce qu'il faut avoir compris à la fin de ce document

1. **La Phase 1 avait une limite structurelle.** TF-IDF compare des mots, pas des intentions. Ajouter plus de données ne pouvait pas résoudre ce problème.
2. **Les embeddings remplacent TF-IDF.** MiniLM transforme les textes en vecteurs basés sur le sens. Deux phrases similaires en intention donnent des vecteurs proches, même sans mot en commun.
3. **FAISS stocke et cherche dans ces vecteurs.** Ce n'est pas du code produit par l'équipe — c'est une bibliothèque de Meta qu'on installe et qu'on utilise. Elle cherche en quelques millisecondes parmi des milliers de vecteurs.
4. **MiniLM vectorise chaque exemple séparément.** Un objet JSON avec 8 exemples produit 8 vecteurs dans FAISS, tous liés à la même réponse. Plus il y a d'exemples, plus la recherche est précise.
5. **metadata.json fait le lien entre les vecteurs et les réponses.** FAISS retourne des numéros — metadata.json dit ce que chaque numéro représente.
6. **Le LLM rédige, il n'invente pas.** Phi-3 Mini reçoit uniquement les réponses trouvées par FAISS. Il ne peut pas produire d'informations qui ne sont pas dans vos fichiers JSON.
7. **RAG est le nom de la stratégie, pas un outil.** Chercher (FAISS) → enrichir (JSON) → générer (LLM) : voilà ce qu'est le RAG.
8. **Django reste dans le projet.** Il garde les feedbacks, les utilisateurs, les statistiques et l'API REST. Ce qui change, c'est uniquement le moteur de recherche et la génération de réponse.
9. **L'interface PWA est remplacée par React.** La PWA HTML/JS vanilla de la Phase 1 laisse place à une application React. Le streaming LLM, les sources et l'historique de conversation sont bien plus faciles à gérer avec des composants et des hooks. L'API Django ne change pas — seule la couche d'affichage évolue.
10. **TF-IDF n'est pas supprimé — il devient le filet de sécurité.** Si le score FAISS est trop bas (question hors-domaine, formulation très inhabituelle), le système bascule automatiquement sur le moteur TF-IDF de la Phase 1. L'étudiant obtient toujours une réponse. **La Phase 2 ajoute de l'intelligence sans rien retirer.**